Pusher

Reverse Engineering (rev)

The binary is a Movfuscator-obfuscated stack puzzle with 9 gated “slots.” You push/pop characters through a menu until a hidden string matches an embedded target. On success, the program reads **./flag.txt** and prints the real flag.

server.py simply runs **./challenge** with stdin/stdout over TCP. The binary reads the real flag from **./flag.txt** next to itself - the handout does not contain the real flag.

Embedded in the binary is a decoy string:

```
OMNICTF{G3T_R3A1_0N_R3M0TE_B0z0_TH1S_1S_NOT_A_HAND0UT}
```

That string is copied into hz_target and used as the strcmp target. You must build that exact 54-character string on the in-memory stack, then trigger the win check.

So I make this script to solve it:

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import re
import socket
import ssl
import subprocess
import sys
import time
from pathlib import Path

HERE = Path(__file__).resolve().parent
TARGET = "OMNICTF{G3T_R3A1_0N_R3M0TE_B0z0_TH1S_1S_NOT_A_HAND0UT}"
assert len(TARGET) == 54
```

```
def pass1() -> list[str]:
    parts: list[str] = []
    for ch in "abc":
        parts += ["1", ch, "2"]
    parts += ["1", "O", "1", "x"]
    for ch in "fgM":
        parts += ["2", "1", ch]
    parts += ["1", "N", "1", "I"]
    return parts


def later(chars: str) -> list[str]:
    parts = ["1", chars[0]]
    ci = 1
    for _ in range(3):
        parts += ["2", "1", chars[ci]]
        ci += 1
    parts += ["1", chars[ci]]
    ci += 1
    for _ in range(3):
        parts += ["2", "1", chars[ci]]
        ci += 1
    while ci < len(chars):
        parts += ["1", chars[ci]]
        ci += 1
    return parts


def later_for_suffix4(suffix4: str) -> list[str]:
    chars = ["x"] * 10
    chars[3] = suffix4[0]
    chars[7] = suffix4[1]
    chars[8] = suffix4[2]
    chars[9] = suffix4[3]
    return later("".join(chars))


def build_moves() -> str:
    parts = pass1()
    for L in range(4, 54):
```

```
        parts += ["2", "2", "2"]
        parts += later_for_suffix4(TARGET[L - 3 : L + 1])
    parts += ["4"]
    return "\n".join(parts) + "\n"


FLAG_RE = re.compile(r"(OMNICTF\{[^}\n]+\}|OmniCTF\{[^}\n]+\}|flag\
{[^}\n]+\})")


def run_local(moves: str) -> str:
    bin_path = HERE / "challenge"
    p = subprocess.run(
        [str(bin_path)],
        input=moves.encode(),
        cwd=str(HERE),
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=120,
    )
    return p.stdout.decode(errors="replace")


def run_remote(host: str, port: int, moves: str, use_ssl: bool) ->
str:
    raw = socket.create_connection((host, port), timeout=30)
    sock: socket.socket = (
        ssl.create_default_context().wrap_socket(raw,
server_hostname=host)
        if use_ssl
        else raw
    )
    sock.settimeout(120)
    time.sleep(0.2)
    try:
        sock.recv(65536)
    except socket.timeout:
        pass
    sock.sendall(moves.encode())
    chunks: list[bytes] = []
    end = time.time() + 120
    while time.time() < end:
```

```
        try:
            data = sock.recv(65536)
        except socket.timeout:
            break
        if not data:
            break
        chunks.append(data)
        if b"Congrats" in data or b"flag" in data.lower():
            sock.settimeout(2)
            try:
                while True:
                    more = sock.recv(65536)
                    if not more:
                        break
                    chunks.append(more)
            except socket.timeout:
                pass
            break
    sock.close()
    return b"".join(chunks).decode(errors="replace")


def main() -> int:
    ap = argparse.ArgumentParser(description="Solve Pusher CTF")
    ap.add_argument("host", nargs="?", help="remote host")
    ap.add_argument("port", nargs="?", type=int, default=1337)
    ap.add_argument("--ssl", action="store_true", help="wrap
connection in TLS")
    ap.add_argument("-o", "--output", type=Path, help="write raw
transcript")
    args = ap.parse_args()

    moves = build_moves()
    (HERE / "solve_input.txt").write_text(moves)

    if args.host:
        print(f"[*] remote {args.host}:{args.port} ssl={args.ssl}",
file=sys.stderr)
        out = run_remote(args.host, args.port, moves, args.ssl)
    else:
        print("[*] local ./challenge", file=sys.stderr)
        out = run_local(moves)
```

```
    if args.output:
        args.output.write_text(out)

    print(out)
    m = FLAG_RE.findall(out)
    real = [f for f in m if f != TARGET]
    if real:
        print(f"[+] flag: {real[-1]}", file=sys.stderr)
        return 0
    if "Congrats" in out:
        print("[+] won, but flag regex missed — see transcript",
file=sys.stderr)
        return 0
    print("[-] no win/flag in output", file=sys.stderr)
    return 1


if __name__ == "__main__":
    raise SystemExit(main())
```